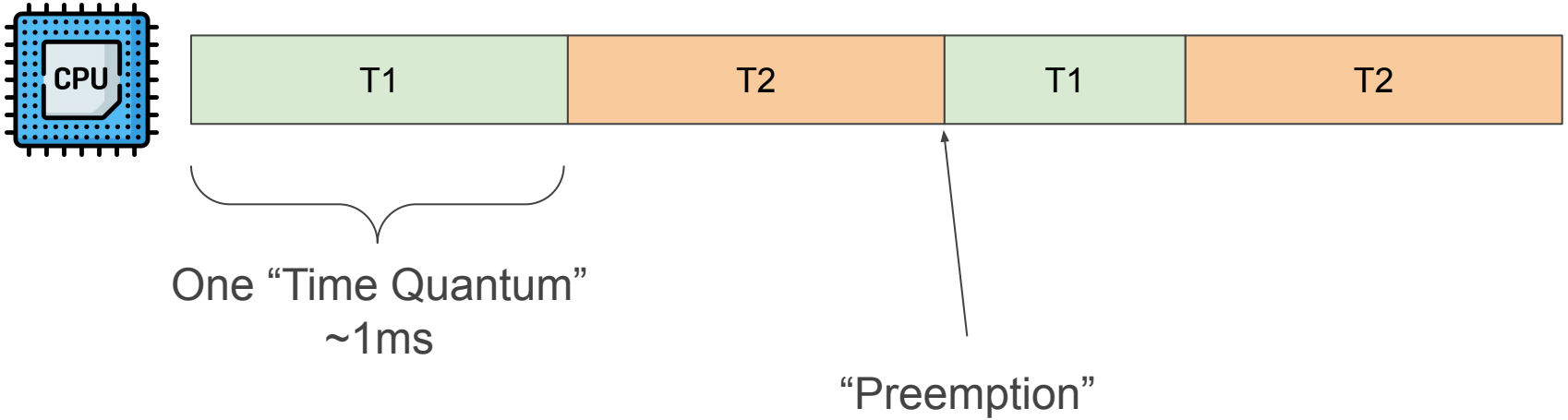
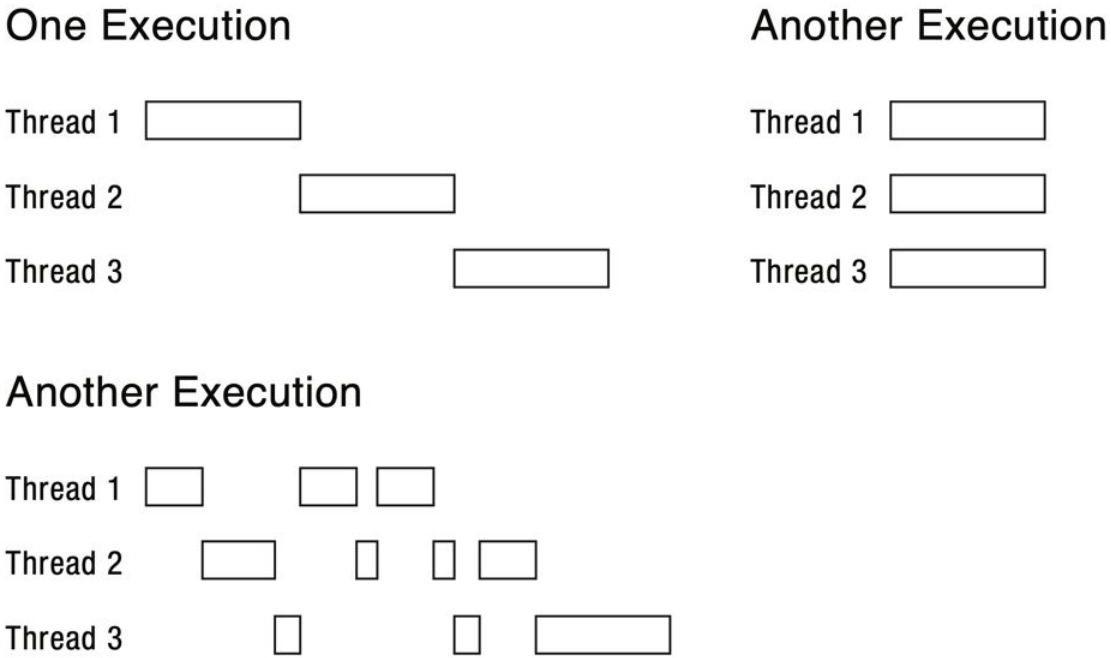
Writing Multithreaded Code

Multithreaded Code: Recap



Multithreaded Code: Recap



Let’s write some parallel code!

Goal: count to 100 as fast as possible.

Reference:

```
int counter;

void main() {

    counter = 0;

    for (int i = 0; i < 100; i++) counter++;

}
```

Let’s write some parallel code!

Goal: count to 100 as fast as possible.

Reference:

```
int counter;

void main() {

    counter = 0;

    for i in (0..100) counter++;

}
```

Let’s write some parallel code!

Thread 0:

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        counter++;

}
```

Thread 1:

```
void main1() {

    for i in (0..50)

        counter++;

}
```

When both main0 and main1 is done, we will read counter

Problem 1: main0 finishes before main1 starts

We didn’t actually get any parallelism!

We want main0 and main1 to run at the same time (“gang scheduling”)

Problem 2: main1 runs before main0 starts

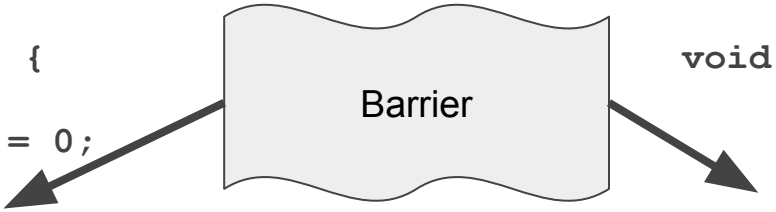
We never cleared the counter to 0!

counter will only count to 50

Problem 2 solution

Thread 0:

```
int counter;

void main0() {

    counter = 0;

    wait();

    for i in (0..50)

        counter++;

}
```



Thread 1:

```
void main1() {

    wait();

    for i in (0..50)

        counter++;

}
```

Barrier

Synchronization primitive

N threads stop at `wait()` and cannot proceed until all N threads arrive

Where do we generally need barriers? Where else do we need a barrier in our program?

Any more problems?

Thread 0:

```
int counter;

void main0() {

    counter = 0;

    wait();

    for i in (0..50)

        counter++;

}
```

Thread 1:

```
void main1() {


    wait();

    for i in (0..50)

        counter++;

}
```

Problem N: We counted somewhere between 50-99!

counter++ is actually 3 distinct operations:

```
load counter, %rax

add $1, %rax

store %rax, counter
```

Problem N Visualization

Core 0:

counter = 98

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

i = 49

Core 1:

```
void main1() {

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

i = 49

Problem N Visualization

Core 0:

counter = 98

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

rax = 99

Core 1:

```
void main1() {

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

i = 49

Problem N Visualization

Core 0:

counter = 98

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

rax = 99

Core 1:

```
void main1() {

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

rax = 99

Problem N Visualization

Core 0:

counter = 99

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

write 99

Core 1:

```
void main1() {

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

rax = 99

Problem N Visualization

Core 0:

counter = 99

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

write 99

Core 1:

```
void main1() {

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

write 99

If this happens every iteration…

We only count to 50 instead of 100!

Any guesses as to what the min/max values of counter are? Can we get to 0? Can we get to 101?

Problem N^2: We counted somewhere between 2-50!

But how can we possibly get a counter value of 2?

Problem N^2 Visualization

Core 0:

```
int counter;
void main0() {
    counter = 0;
    for i in (0..50)
        load counter, %rax
        add $1, %rax
        store %rax, counter
}
```

Core 1:

```
void main1() {

    for i in (0..50)
        load counter, %rax
        add $1, %rax
        store %rax, counter
}
```

Problem N^2 Visualization

Core 0:

counter = 0

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

i = 0

Core 1:

```
void main1() {

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

i = 0

Problem N^2 Visualization

Core 0:

counter = 0

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

i = 0

Core 1:

```
void main1() {

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

rax = 1

Problem N^2 Visualization

Core 0:

counter = 49

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

i = 49

Core 1:

```
void main1() {

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

rax = 1

Problem N^2 Visualization

Core 0:

counter = 1

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

i = 49

Core 1:

```
void main1() {

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

write 1

Problem N^2 Visualization

Core 0:

counter = 1

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

rax = 2

Core 1:

```
void main1() {

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

i = 1

Problem N^2 Visualization

Core 0:

counter = 50

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

rax = 2

Core 1:

```
void main1() {

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

… write 50

Problem N^2 Visualization

counter = 2

Core 0:

```
int counter;

void main0() {

    counter = 0;

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

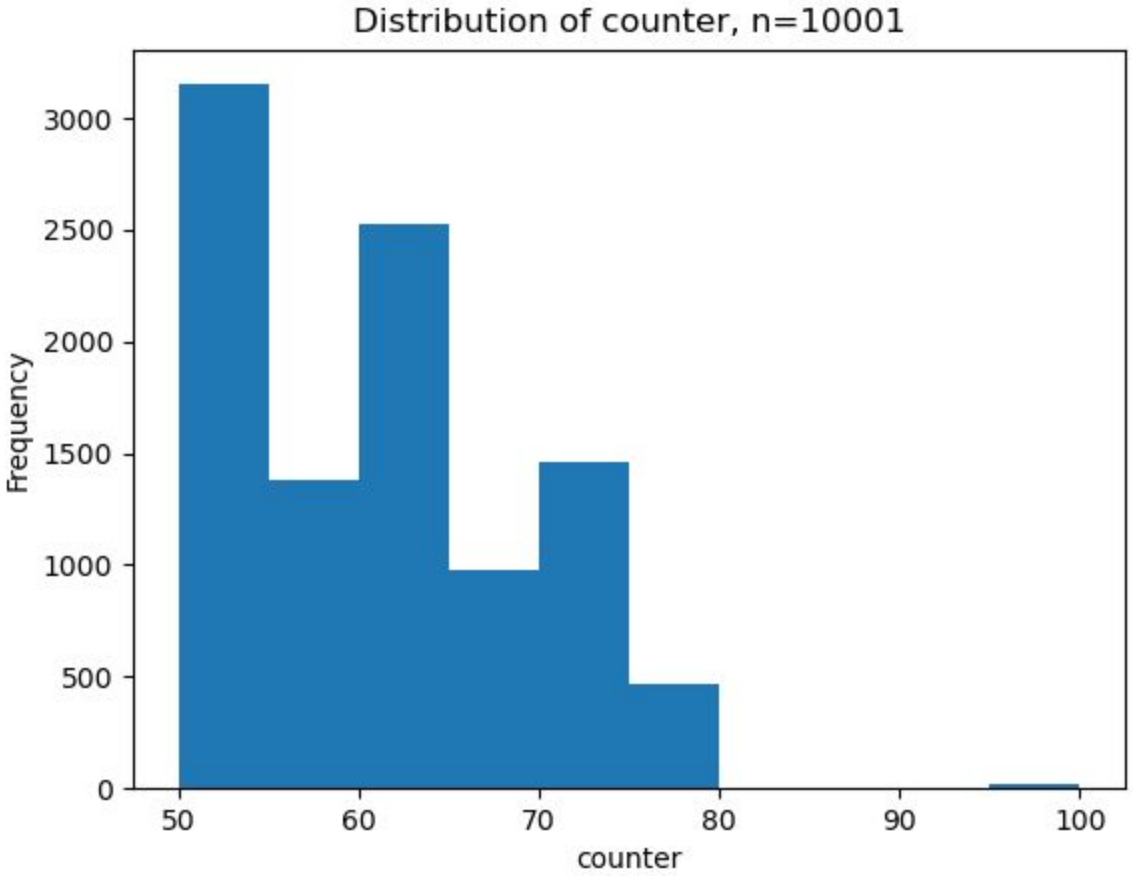
write 2

Core 1:

```
void main1() {

    for i in (0..50)

        load counter, %rax

        add $1, %rax

        store %rax, counter

}
```

Live demo

Results


Distribution of counter, n=10001
Frequency
3000
2500
2000
1500
1000
500
0
50
60
70
80
90
100
counter

Why is 100 slightly more common than 90?

In theory, each number should be less likely to occur than the last

In practice, the OS may decide to only run thread 0 and not thread 1 right after our wait() call, so the code isn’t running in parallel

Why don’t we see 49?

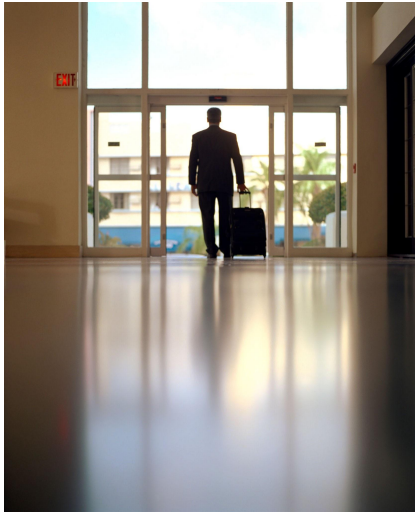
In theory, any number 2-49 is possible

In practice, getting 49 itself is incredibly rare

- The cores aren’t starting at exactly the same time because the loop is so fast
  - Sub 50 requires a precise overlap of execution

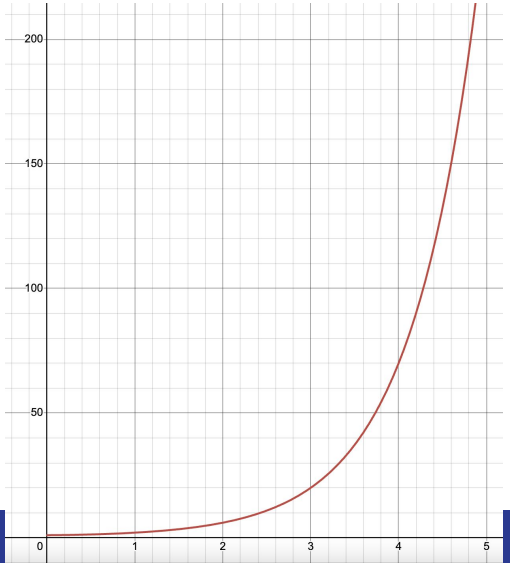
Interleavings

The root of the problem is that there are a bajillion interleavings between the instructions run on threads 0 and 1!



Interleavings

- Threads can interleave instructions in any and every possible ordering
  - In a 2-thread program with N instructions, there are O(2^N) possible executions
    - 2N choose N => (2N)!/(N!^2)
  - Most of those are probably bug-free, but even 1 buggy means your code will sporadically fail



Thread 0: `a; b;` Thread 1: `c; d;`

3 threads

All possible interleavings

0	a	b		
1			c	d

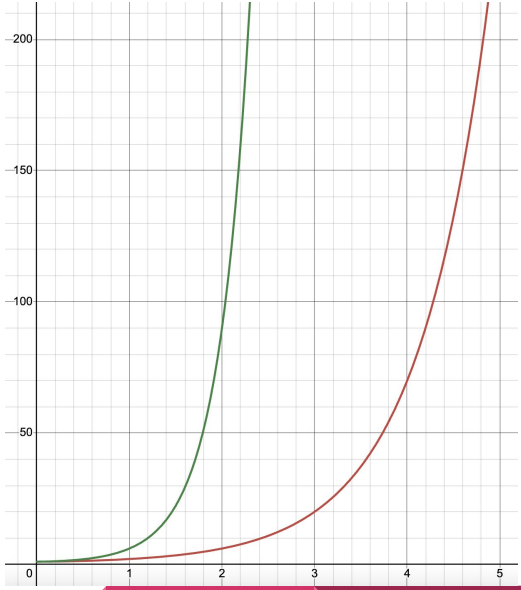
0	a		b	
1		c		d

0	a			b
1		c	d	

0			a	b
1	c	d		

0		a		b
1	c		d	

0		a	b	
1	c			d



Time

Wait, why can’t a execute in parallel with c?

a: counter = 1

c: counter = 2

At the end of the day, counter *must* be either 1 or 2

So one must run before the other!

What went wrong?

Multiple reads and writes to data without synchronization

“data race”, “unsafe” memory usage

What is “safe”? multiple readers OR one writer, but NOT both, between synchronization points

Data Race

2 instructions from different threads access the same memory

at least one is a write

no synchronization mandating an order between the accesses

e.g.: counter++ on two threads!

How can we fix it?

In practice, there are situations where we need multiple readers and writers, but…

- we can access different memories sometimes
- we can synchronize them

How can we fix it?

2 instructions from different threads access the same memory

- access different memory, then combine the local counters at the end

no synchronization mandating an order between the accesses

- add <u>synchronization</u> around problematic parallel accesses
  - e.g. Critical Section (via locks), Channels
  - Barriers are awkward here, since we want one-thread-at-a-time

Barriers to the rescue?

Thread 0:

```
void main0() {
    for i in (0..50)
        counter++;
        wait();
        <let main1 go>
        wait();
}
```

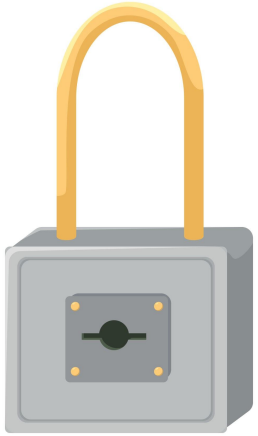
Thread 1:

```
void main1() {
    for i in (0..50)
        <let main0 go>
        wait();
        counter++;
        wait();
}
```

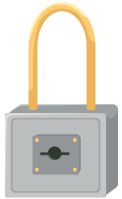
Locks

lock / acquire: “grab” the lock, don’t let anyone else grab the lock while I hold it

unlock / release: let go of the lock, now someone else can lock/acquire it



Synchronization via locks



Thread 0:

```
for i in (0..50)
    lock L
    load counter, %rax
    add $1, %rax
    store %rax, counter
    unlock L
```

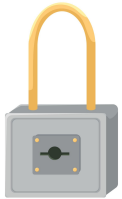
Thread 1:

```
for i in (0..50)
    lock L
    load counter, %rax
    add $1, %rax
    store %rax, counter
    unlock L
```

Critical Section

One thread at a time!

Locks are powerful



Thread 0:

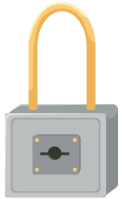
```
RedBlackTree my_tree;

void main0() {

    my_tree.insert(0)

}
```

Thread 1:

```
void main1() {

    my_tree.insert(1)

}
```

Problem: 2 writers to a shared object

Locks are powerful



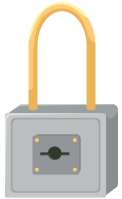
Thread 0:

```
RedBlackTree my_tree;

void main0() {

    L.lock();

    my_tree.insert(0)

    L.unlock();

}
```

Thread 1:

```
Lock L;

void main1() {

    L.lock();

    my_tree.insert(1)

    L.unlock()

}
```

Locks are dangerous



What if you forget to call unlock()?

What if you try and lock a lock you already have acquired?

Locks are slow if there’s a lot of contention between cores

Locks are hard to debug

There are many more problems we’ll get into

Solution 1: don’t share memory, lock very little

Thread 0:

```
void main0() {
    local_counter = 0;
    for i in (0..50)
        local_counter++;
    L.lock();
    counter += local_counter;
    L.unlock();
}
```

Thread 1:

```
void main1() {
    local_counter = 0;
    for i in (0..50)
        local_counter++;
    L.lock();
    counter += local_counter;
    L.unlock();
}
```

Other alternatives to locks

Channels (functional decomposition)

- Dedicate thread 0 to reading “increment” messages on shared channels
- Dedicate threads 1 through N to sending increment messages

Lock-free and Wait-free algorithms

- Extremely difficult

Don’t Share Mutable State

Many flavors of locks

- SpinLock, InterruptSafeLock, Mutex / BlockingLock, RWLock, SeqLock

Do locks solve all problems?

```
void main0() {
    lock(L);
    my_num = 3;
    unlock(L);
}
```

```
void main1() {
    lock(L);
    my_num = 4;
    unlock(L);
}
```

<u>Race Condition</u>
When there's an undefined ordering of execution that affects the end result

What if we really want my_num = 3, then = 4?

```
int done = 0;

void main0() {

    my_num = 3;

    done = 1;

}
```

```
void main1() {

    while (done != 1) {}

    assert(my_num == 3);

    my_num = 4;

}
```

What if we really want my_num = 3, then = 4?

```
int done = 0;

void main0() {

    my_num = 3;

    done = 1;

}
```

```
void main1() {

    while (done != 1) {}

    assert(my_num == 3);

    my_num = 4;

}
```

%rax

Compiler optimizations!

What if we really want my_num = 3, then = 4?

```
volatile int done = 0;

void main0() {

    my_num = 3;

    done = 1;

}
```

```
void main1() {

    while (done != 1) {}

    assert(my_num == 3);

    my_num = 4;

}
```

What if we really want my_num = 3, then = 4?

```
volatile int done = 0;

void main0() {
    my_num = 3;
    done = 1;
}
```

```
void main1() {
    while (done != 1) {}
    assert(my_num == 3);
    my_num = 4;
}
```

Compiler reordering!

What if we really want my_num = 3, then = 4?

```
volatile int done = 0;

void main0() {
    my_num = 3;
    compiler_barrier();
    done = 1;
}
```

```
void main1() {
    while (done != 1) {}
    assert(my_num == 3);
    my_num = 4;
}
```

What if we really want my_num = 3, then = 4?

```
volatile int done = 0;

void main0() {
    my_num = 3;
    compiler_barrier();
    done = 1;
}
```

```
void main1() {
    while (done != 1) {}
    assert(my_num == 3);
    my_num = 4;
}
```

CPU reorders instructions!!

What if we really want my_num = 3, then = 4?

```
volatile int done = 0;

void main0() {
    my_num = 3;
    memory_fence();
    compiler_barrier();
    done = 1;
}
```

```
void main1() {
    while (done != 1) {}
    assert(my_num == 3);
    my_num = 4;
}
```

Multithreaded code is really hard to write!

- Must be thinking about compiler, CPU, caches…
- Proper synchronization is crucial to preventing:
  - Data races
  - Race conditions
- How should you share data between one thread and another?
  - Use synchronization primitives!
  - Stick to the standard library synchronization primitives
- Everything we talked about still applies with only 1 core!